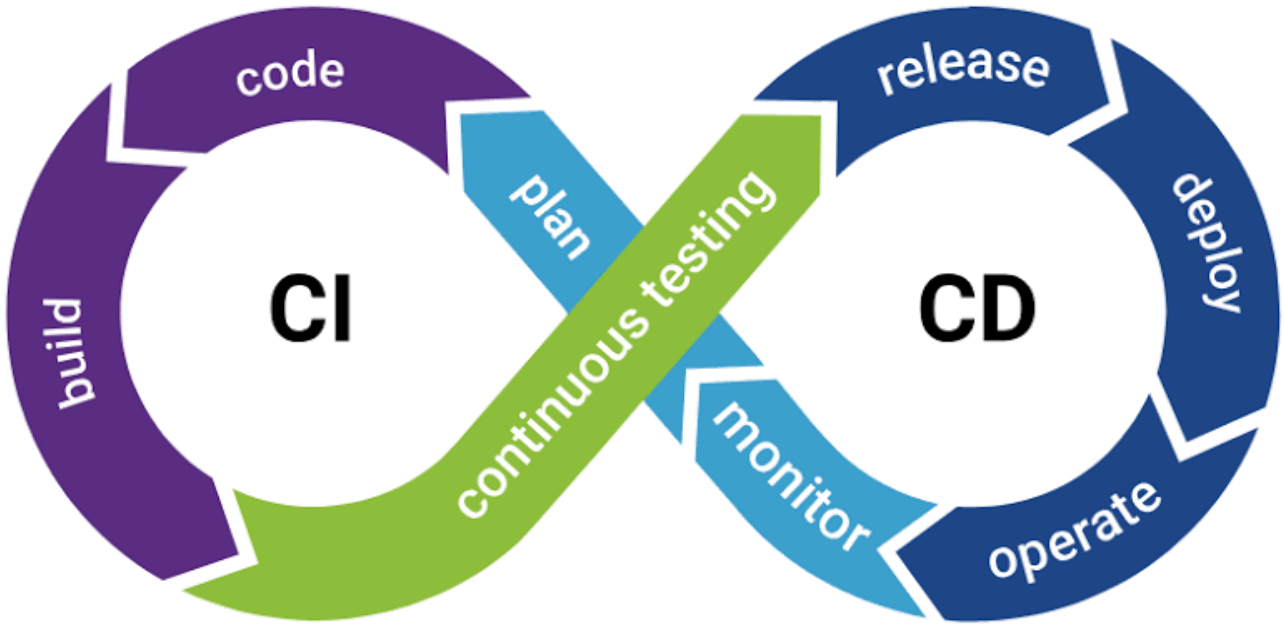


Pipeline Terraform

CI/CD alapok

CUBIX
INSTITUTE OF TECHNOLOGY

Miért kell?


code
plan
release
deploy
build
CI
continuous testing
CD
monitor
operate

Mit várunk el

- Automatizáció
- Egységes futtatkókörnyezet
- Auditálhatóság
- Policy kikényszerítés

Imperatív vs. Deklaratív CD eszközök

Imperatív

- Az utasításokat mi definiáljuk (amit nem, azt nem)
- A végeredmény változására nem reagál
- Pipeline-ok, pl. GitHub actions

Deklaratív

- Mi a kívánt állapotot írjuk le
- Lehetőség van drift detectionre
- Folyamatosan újra előállítható
- pl.: ArgoCD, Flux



Pipeline Terraform

Pipeline-ok, GitHub Actions

CUBIX
INSTITUTE OF TECHNOLOGY

Mi az a pipeline?

- Automatizált lépéssorozat
  - `terraform init`
  - `terraform apply`
- Ellenőrizhető és ismételhető folyamat
- Triggerek
  - Git push
  - Pull request
  - Kézi indítás

GitHub Actions

- Event = Trigger
- Workflow = Pipeline
- Job = Párhuzamosítás alapjai
- Step = Konkrét kis lépés



Egy példa

- GitHub repó: `cubix-pe-on-air`
- Írjuk ki az időt
- Futtassuk
- Nézzünk bele



Pipeline Terraform

GitHub és Azure integráció

CUBIX
INSTITUTE OF TECHNOLOGY

Miért?

- A terraformot pipeline-ból akarjuk futtatni
- Kell hogy legyen joga Azure-ben dolgozni

Hogyan?

- Hasonlóan a 2. héten tanultakhoz
- A GitHub is Microsoft termék
- Workload identity federation
- Jogokat is kell majd adni

Csináljuk

1. Repó: Múlt heti terraform kód repó
2. Azure Managed identity
   - Resource group: `course-baseline`
   - Név: `github-repos`
3. Federated Credentials
4. Repóban beállítjuk az identitást
5. Adjunk jogokat

Próbáljuk ki

- `az group list`
- Biztos van legalább egy resource groupunk



Pipeline Terraform

Terraform futtatása GitHubon

CUBIX
INSTITUTE OF TECHNOLOGY

Előfeltételek

- GitHub Actions tud authentikálni
- Owner az előfizetésen
- Data contributor a tfstate blob store-on



Cél

- Masteren/mainen lehessen csak `terraform apply`
- Pull request csak akkor mehessen be, ha fut `terraform plan`
- Kötelező legyen a semi-linear history

Design

- 2 job
  - plan
  - apply
- 2 trigger
  - `master` / `main`
  - `pull_request`

- GitHub ruleset
  - Kötelező check
  - Semi-linear history



Pipeline Terraform

Helm deployment pipeline-ban

CUBIX
INSTITUTE OF TECHNOLOGY

Lehetőségek

Direkt felhasználás

- Az összes CRUD műveletet újraimplementálni
- A terraform kimenetet használni kell a helmnél

Terraformban

- Elég a terraformot jól kezelni
- Más erőforrások direkt elérhetők
- Ezt választjuk

CUBIX INSTITUTE OF TECHNOLOGY

Nézzük meg az ingress controlleren

- Kell a helm provider
- Kell neki egy kubeconfig
- Az első heti parancsot konvertáljuk

```
$ helm upgrade --install traefik traefik \
  --repo https://traefik.github.io/charts \
  --version 41.2.0 \
  --namespace ingress-controller \
  --create-namespace \
  --set service.annotations."service\.beta\.kubernetes\.io/azure-load-balancer-health-probe-request
  --set ports.web.http.redirections.entryPoint.to=websecure \
  --set ports.web.http.redirections.entryPoint.scheme=https \
  --set ports.web.http.redirections.entryPoint.permanent=true
```

CUBIX INSTITUTE OF TECHNOLOGY

Kubernetes resource-ok

- A helm provider megengedő
- A kubernetes provider nem
- Csel: `raw` chart @ `--repo=https://bedag.github.io/helm-charts`
- Let’s Encrypt Clusterissuer-hez kelleni fog

Feladat

- Változtasd meg az AKS default nodepool típusát
- Cseréld le az ingress controllert
- Ugye mennyivel egyszerűbb már most?
- Azért csak óvatosan!



Pipeline Terraform

Problémák a pipeline-nal

CUBIX
INSTITUTE OF TECHNOLOGY

A kerék újrafeltalálása

- A terraform csak akkor fut, amikor kérjük
- Minden operációra készülni kell a pipeline-ban
- Nincs drift detection
- A hibakezelés is a mi dolgunk

Miért maradunk itt terraformban

- Az "élő" funkciók fizetősek
- Terraform Cloud

Több környezet támogatása

- A backendbe leírtuk a statefájl nevét
- Más változó → Más kívánt állapot
- A backendet is dinamizálni kell
- Megoldás: `terraform workspace`

Több környezet támogatása

Biztonsági intézkedés

```
resource "terraform_data" "workspace_guard" {
  input = var.env_name

  lifecycle {
    precondition {
      condition     = terraform.workspace == var.env_name
      error_message = "Workspace must match env: ${var.env_name} != ${var.env_name}"
    }
  }
}
```



Pipeline Terraform

Házi feladat

CUBIX
INSTITUTE OF TECHNOLOGY

Feladat

- A korábban megírt terraform kódot futtasd GitHub Actionsből
- Egészítsd ki a terraform kódot, hogy a kubernetesen belüli erőforrásokat is deployolja
- Állítsd be, hogy pull requestre fusson terraform plan és validate

Feladat

- Állítsd be, hogy
  - Csak pull requesten keresztül mehessen be kód mainbe
  - A terraform plan és validate legyen feltétele a merge-nek
  - Várd el, hogy csak friss branchről lehessen merge-elni
- Mainre merge-kor fusson terraform apply

Beadandó

- A környezetigénylési leírás frissítve
- A terraform kód egy github repóban (repó link) megtagelve (`week4-pipeline-tf`)
- Pull request, amiben a dev környezetet "megigényled"
  - Legyen egy merge-elve
  - Legyen egy nyitva